

✓ CI com GitHub Actions — Bella Tavola 🍷

Parte 1: Fundamentos de CI e GitHub Actions

Como usar este caderno?

Este documento é seu guia de aprendizagem para a Semana 3 — Parte 1. Leia cada seção, estude o código de referência e implemente os exercícios no seu ambiente local.

Ao contrário das semanas anteriores, a maior parte do trabalho aqui acontece **fora do Python** — no repositório GitHub, nos arquivos YAML e nos logs do Actions. Tenha o navegador aberto no seu repositório enquanto trabalha.

Os gabaritos estão nas células colapsadas logo após cada exercício — tente resolver antes de abrir.

O Contexto

A API do Bella Tavola está funcionando. O modelo de predição está publicado no Hugging Face Hub. Mas toda vez que alguém do time faz uma alteração no código, a pergunta é a mesma:

"Será que quebrou alguma coisa?"

Hoje você vai aprender a responder essa pergunta automaticamente — a cada commit, a cada pull request, sem precisar lembrar de rodar nada manualmente.

O que é essencial, recomendado e desafio?

Ao longo do caderno, as atividades aparecem com três níveis:

Essencial: o mínimo que você precisa conseguir fazer para completar a etapa.

Recomendado: amplia a qualidade da solução.

Desafio: aprofunda a solução do ponto de vista de engenharia.

Pré-requisitos

Antes de começar, confirme que você já tem:

- A API do Bella Tavola funcionando localmente (Semanas 1 e 2)
- O modelo publicado no Hugging Face Hub (Semana 2)
- Conta no GitHub com o projeto em um repositório
- Git configurado localmente

✓ Verificação rápida do ambiente

Antes de criar qualquer arquivo de CI, confirme que sua aplicação funciona localmente do jeito que o pipeline vai esperar. Abra um terminal e rode:

```
uvicorn main:app --reload
```

Em outro terminal, teste se a API responde:

```
curl http://localhost:8000/
```

Se o comando acima falhar, pare aqui e resolva primeiro. O GitHub Actions vai executar exatamente esse mesmo processo em uma máquina remota. Se não funciona localmente, não vai funcionar no pipeline.

Pontos comuns que precisam de ajuste antes de continuar:

- **O app não está em `main.py`?** Anote o caminho correto — você vai precisar dele no workflow. Por exemplo, se o arquivo principal for `app/server.py` com `app = FastAPI()`, o comando seria `uvicorn app.server:app`.
- **Sua API não tem rota `/`?** Use uma rota que exista de fato, como `/docs`, `/health` ou `/pratos`. Anote qual rota você vai usar para testar.
- **Sua API depende de um arquivo `.env`?** Essas variáveis precisarão ser configuradas separadamente no GitHub Actions — veremos isso na Parte 3.

```
#  Anote aqui antes de continuar:
```

```
# Comando para iniciar sua API:
```

```
# uvicorn __:app
```

```
# Rota que existe e retorna 200 na sua API:
```

```
# http://localhost:8000/___
```

```
# Sua API depende de variáveis de ambiente além de HF_TOKEN? (sim/não)
# Se sim, quais:
```

✓ BLOCO 1

Objetivo: Entender por que automação importa, o que é CI/CD e como esses conceitos se aplicam ao contexto do Bella Tavola. Ao final deste bloco, você conseguirá explicar a diferença entre CI e CD e identificar em qual etapa do ciclo de vida do software cada prática atua.

✓ Conceitos-chave do Bloco 1

O problema sem automação

Imagine o seguinte cenário: você e mais duas pessoas do time estão trabalhando na API do Bella Tavola. Uma pessoa altera a rota `POST /pedidos`. Outra refatora os modelos Pydantic. Você adiciona o endpoint `/predict`.

No final do dia, alguém faz o merge de tudo. A API quebra. Mas qual mudança causou o problema? E quando exatamente ela foi introduzida?

Sem automação, a resposta envolve: testar manualmente, comparar versões, adivinhar. Com CI, a resposta chega em minutos, direto no commit que causou o problema.

O que é CI?

CI significa **Integração Contínua** (*Continuous Integration*). É a prática de integrar as mudanças de código de todos os membros do time com frequência — idealmente várias vezes ao dia — e verificar automaticamente se essa integração não quebrou nada.

A verificação automática acontece por meio de um **pipeline**: uma sequência de etapas que roda a cada novo commit.

Um pipeline típico de CI faz:

1. Baixa o código novo
2. Instala as dependências
3. Roda os testes automatizados
4. Reporta o resultado:  passou ou  falhou

Se falhou, o time sabe imediatamente — e sabe exatamente em qual commit.

O que é CD?

CD pode significar duas coisas relacionadas:

Entrega Contínua (*Continuous Delivery*): após o pipeline de CI passar, o código está automaticamente preparado e empacotado para ser publicado — mas um humano ainda decide quando colocar em produção.

Implantação Contínua (*Continuous Deployment*): vai um passo além. Se o CI passou, o código vai automaticamente para produção, sem intervenção humana.

```
Desenvolvedor faz commit
    ↓
  CI roda testes ← você vai construir isso hoje
    ↓
  CD empacota artefato
    ↓
  CD publica em produção
```

Neste caderno, o foco é **CI**. O CD envolve infraestrutura de deploy que varia muito entre projetos.

Por que CI é especialmente importante em projetos de ML?

Em projetos de software tradicional, uma mudança no código pode quebrar uma funcionalidade. Em projetos de ML, uma mudança pode:

- Quebrar uma rota da API
- Alterar silenciosamente o formato esperado pelo modelo
- Mudar a ordem das features e gerar previsões erradas sem nenhum erro aparente
- Fazer o carregamento do modelo falhar apenas em ambiente de produção

CI não resolve todos esses problemas, mas força o time a tornar o comportamento esperado **explícito** — em forma de testes — e a verificar esse comportamento a cada mudança.

GitHub Actions

É a ferramenta de CI/CD integrada ao GitHub. Você descreve seu pipeline em um arquivo YAML dentro do repositório, e o GitHub executa esse pipeline automaticamente em resposta a eventos — como um `push` ou um `pull request`.

A grande vantagem: a infraestrutura é do GitHub. Você não precisa configurar servidor, instalar agente, nem gerenciar máquinas. Basta o arquivo YAML no lugar certo.

✓ Exercício 1.1 — Diagnóstico: o que pode quebrar?

Nível: Essencial

Conceito: Pensar em termos de testes antes de escrever código de automação.

Antes de automatizar qualquer coisa, é preciso saber **o que vale a pena verificar**.

✓ Sua tarefa

Sem escrever código, liste pelo menos **6 situações** em que uma mudança no código do Bella Tavola poderia quebrar algo silenciosamente — ou seja, sem gerar um erro óbvio imediato.

Para refletir: Das situações que você listou, quais seriam detectadas por um teste automatizado? Quais exigiriam monitoramento em produção?

 Escreva sua lista aqui como comentários:

```
# 1.  
# 2.  
# 3.  
# 4.  
# 5.  
# 6.
```

 Gabarito

```
# @title  
# Situações em que mudanças podem quebrar algo silenciosamente:  
  
# 1. Alguém renomeia um campo no PratoInput (ex: 'preco' → 'valor')  
#   → A rota POST /pratos começa a retornar 422 para todos os clientes  
  
# 2. A ordem das features no endpoint /ml/predict é alterada  
#   → O modelo recebe dados na ordem errada e faz previsões incorretas  
#   → Não gera erro — só prediz errado
```

```
# 3. Um validador @field_validator é removido acidentalmente
#   → Preços negativos voltam a ser aceitos

# 4. A rota GET /pratos para de aplicar o filtro apenas_disponiveis
#   → Pratos indisponíveis aparecem no cardápio

# 5. O model.pkl no Hugging Face Hub é substituído por uma versão
#   treinada com features diferentes
#   → O carregamento funciona, mas as previsões ficam erradas

# 6. Um import circular é introduzido ao reorganizar os routers
#   → A API inteira falha ao iniciar, mas só em certos ambientes

# ---
# Para refletir:
# Situações 1, 3, 4 e 6 → detectáveis com testes automatizados
# Situação 2 → detectável com teste de contrato de features
# Situação 5 → exige monitoramento de métricas em produção (data drift)
```

✓ Exercício 1.2 — CI vs CD na prática

Nível: Essencial

Conceito: Classificar práticas reais em CI ou CD, consolidar o entendimento conceitual.

✓ Sua tarefa

Classifique cada item abaixo como **CI**, **CD (Entrega)** ou **CD (Implantação)**. Anote também uma justificativa curta.

```
#  Classifique cada item:

# 1. A cada pull request, os testes de pytest rodam automaticamente.
#   Classificação:
#   Por quê:

# 2. Após os testes passarem, um novo container Docker é gerado
#   e armazenado no registry, aguardando aprovação para ir a produção.
#   Classificação:
#   Por quê:

# 3. Quando o branch main recebe um merge, a API é publicada
#   automaticamente no servidor de produção sem nenhuma aprovação manual.
#   Classificação:
#   Por quê:
```

```
# 4. O pipeline verifica se o requirements.txt está atualizado
# em relação às importações do código.
# Classificação:
# Por quê:

# 5. Após o merge, o model.pkl mais recente é baixado do Hugging Face
# Hub e os testes de predição são executados com ele.
# Classificação:
# Por quê:
```

💡 Gabarito

```
# @title
# 1. A cada pull request, os testes de pytest rodam automaticamente.
# Classificação: CI
# Por quê: Verifica a integração do código novo antes do merge.

# 2. Após os testes passarem, um novo container Docker é gerado
# e armazenado no registry, aguardando aprovação para ir a produção.
# Classificação: CD – Entrega Contínua
# Por quê: O artefato está pronto para deploy, mas um humano decide quando.

# 3. Quando o branch main recebe um merge, a API é publicada
# automaticamente no servidor de produção sem nenhuma aprovação manual.
# Classificação: CD – Implantação Contínua
# Por quê: Não há intervenção humana entre o merge e a produção.

# 4. O pipeline verifica se o requirements.txt está atualizado
# em relação às importações do código.
# Classificação: CI
# Por quê: É uma verificação de qualidade que roda a cada integração.

# 5. Após o merge, o model.pkl mais recente é baixado do Hugging Face
# Hub e os testes de predição são executados com ele.
# Classificação: CI
# Por quê: Verifica que o modelo atual ainda passa nos testes esperados.
# (Poderia ser parte de CD se fosse o gatilho para o deploy.)
```

✓ BLOCO 2

Objetivo: Dominar a sintaxe YAML no contexto do GitHub Actions e entender a anatomia completa de um arquivo de workflow. Ao final deste bloco, você conseguirá ler qualquer workflow do GitHub Actions e explicar o que cada parte faz — e criar o primeiro pipeline funcional do Bella Tavola.

✓ Conceitos-chave do Bloco 2

O que é YAML?

YAML é um formato de arquivo usado para configuração. O GitHub Actions usa YAML para descrever pipelines. Antes de escrever workflows, é preciso entender a sintaxe.

Regras fundamentais:

Indentação define hierarquia — use sempre espaços, nunca tabs.

```
nivel_1:
  nivel_2:
    nivel_3: valor
```

Dois tipos principais de estrutura:

```
# Mapeamento (chave: valor)
nome: Bella Tavola
versao: 1.0.0
ativo: true

# Lista (itens com -)
categorias:
  - pizza
  - massa
  - sobremesa
```

Tipos de valores e strings multilinha

```
texto: "Bella Tavola"      # string
numero_inteiro: 42         # int
numero_decimal: 3.14       # float
verdadeiro: true          # bool
nulo: null                 # null
lista_inline: [a, b, c]    # lista em uma linha
mapa_inline: {a: 1, b: 2}  # mapa em uma linha
```

Strings multilinha:


```
# Literal (preserva quebras de linha)
script: |
  pip install -r requirements.txt
  pytest tests/ -v

# Dobrado (quebras de linha viram espaços)
descricao: >
  Este é um texto longo
  que será tratado como
  uma linha só.
```

Anatomia de um workflow do GitHub Actions

Um workflow é um arquivo `.yaml` dentro da pasta `.github/workflows/` do seu repositório. Ele tem quatro partes principais:

```
# 1. NOME do workflow (aparece na aba Actions do GitHub)
name: CI Pipeline

# 2. GATILHOS – quando este workflow deve rodar
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

# 3. JOBS – grupos de etapas que rodam em uma máquina
jobs:
  test:
    runs-on: ubuntu-latest
    # nome do job (você escolhe)
    # sistema operacional

# 4. STEPS – etapas executadas em sequência dentro do job
steps:
  - name: Baixar o código
    uses: actions/checkout@v4

  - name: Configurar Python
    uses: actions/setup-python@v5
    with:
      python-version: "3.11"
```

```
- name: Instalar dependências
  run: pip install -r requirements.txt

- name: Rodar testes
  run: pytest tests/ -v
```

Detalhando os gatilhos (on)

```
on:
  push:                # roda quando alguém faz push
    branches: [main]   # apenas no branch main

  pull_request:        # roda quando um PR é aberto ou atualizado
    branches: [main]   # apenas em PRs direcionados ao branch main

  workflow_dispatch:    # permite rodar manualmente pela interface do GitHub
```

Uma observação importante sobre `pull_request`: quando você especifica `branches: [main]`, o workflow roda em PRs **direcionados** ao main — ou seja, quando alguém quer fazer merge de qualquer branch para o main. Se omitir `branches`, o gatilho se aplica a PRs direcionados a qualquer branch.

Detalhando os jobs

Um workflow pode ter múltiplos jobs. Por padrão, eles rodam em paralelo. Para forçar sequência:

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps: [...]

  test:
    runs-on: ubuntu-latest
    needs: build          # só começa depois que 'build' terminar com sucesso
    steps: [...]
```

Detalhando os steps — uses vs run

Cada step é uma unidade de trabalho. Pode usar uma action pronta ou rodar um comando shell:

```
steps:
  # Usando uma action pronta (uses)
  - name: Baixar o código
    uses: actions/checkout@v4

  # Rodando um comando shell (run)
  - name: Instalar dependências
    run: pip install -r requirements.txt

  # Rodando múltiplos comandos
  - name: Verificar ambiente
    run: |
      python --version
      pip --version
      pip list
```

`uses` referencia uma **action** — um bloco de lógica reutilizável publicado no GitHub Marketplace. `actions/checkout@v4` baixa o código do repositório. `actions/setup-python@v5` instala o Python na versão que você especificar.

`run` executa comandos shell diretamente na máquina do runner, exatamente como você faria no terminal.

O runner

O runner é a máquina virtual onde o pipeline roda. O GitHub oferece runners gratuitos:

Runner	Sistema
<code>ubuntu-latest</code>	Linux (Ubuntu)
<code>windows-latest</code>	Windows
<code>macos-latest</code>	macOS

Para a maioria dos projetos Python, `ubuntu-latest` é a escolha padrão — é mais rápido e tem mais ferramentas pré-instaladas.

Variáveis de ambiente

Você pode passar variáveis de ambiente para qualquer step:

```
steps:
  - name: Rodar com variável de ambiente
    run: python script.py
  env:
    AMBIENTE: producao
    DEBUG: "false"
```

Na Parte 3, veremos como passar informações sensíveis — como tokens de API — de forma segura usando **secrets**. Por enquanto, não precisamos disso.

✓ Exercício 2.1 — Lendo um workflow

Nível: Essencial

Conceito: Interpretar um arquivo de workflow existente, identificar cada componente.

Analise o workflow abaixo e responda às perguntas nos comentários.

```
name: Bella Tavola – Verificação Completa

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  qualidade:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Instalar dependências
        run: |
          pip install --upgrade pip
          pip install -r requirements.txt

      - name: Verificar formatação
```

```
run: black --check .

- name: Rodar testes
  run: pytest tests/ -v --tb=short

relatorio:
  runs-on: ubuntu-latest
  needs: qualidade
  if: github.ref == 'refs/heads/main'
  steps:
    - uses: actions/checkout@v4
    - name: Gerar relatório
      run: python scripts/gerar_relatorio.py
```

 Responda às perguntas:

- # 1. Em quais situações este workflow é disparado?
- # 2. Quantos jobs existem? Eles rodam em paralelo ou em sequência?
- # 3. O job 'relatorio' sempre roda? Quando ele não roda?
- # 4. O que faz o --tb=short no comando do pytest?
- # 5. O step "Verificar formatação" vai falhar se o código não estiver formatado com black. Isso é desejável? Por quê?

Gabarito

```
# @title
# 1. O workflow é disparado em três situações:
#   - Push para o branch 'main'
#   - Push para o branch 'develop'
#   - Pull request direcionado ao branch 'main'
#   (PRs direcionados a 'develop' não disparam o workflow)

# 2. Existem 2 jobs: 'qualidade' e 'relatorio'.
#   Eles rodam em SEQUÊNCIA – 'relatorio' tem 'needs: qualidade',
#   então só começa quando 'qualidade' terminar com sucesso.

# 3. O job 'relatorio' NÃO roda em pull requests nem em pushes
#   para o branch 'develop'. A condição
#   'if: github.ref == refs/heads/main' garante que ele só
#   executa quando o push é diretamente para o branch main.

# 4. --tb=short exibe um traceback resumido quando um teste falha.
```

```
# O padrão (--tb=long) mostra mais detalhes.
# Em pipelines, o resumo costuma ser suficiente para identificar
# o problema sem poluir demais os logs.

# 5. Sim, é desejável. Formatação consistente reduz ruído nos diffs
# do Git – mudanças de estilo aparecem misturadas com mudanças reais
# e dificultam a revisão de código. Automatizar a verificação garante
# que ninguém 'esquece' de formatar antes de fazer o commit.
```

✓ Exercício 2.2 — Corrigindo um YAML inválido

Nível: Essencial

Conceito: Identificar erros de sintaxe e estrutura YAML.

O YAML abaixo tem **4 erros de sintaxe ou estrutura**. Encontre e corrija todos antes de ver o gabarito.

```
name: CI Bella Tavola
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Instalar dependencias
        run: |
          pip install -r requirements.txt

      - name: Rodar testes
        run: pytest tests/ -v
        env:
          HF_TOKEN: $secrets.HF_TOKEN
```

 Liste os 4 erros que você encontrou:

```
# Erro 1:
# Erro 2:
# Erro 3:
# Erro 4:
```

💡 Gabarito

```
# @title
# ERRO 1: 'runs-on' não está indentado dentro de 'test'
# Errado:
#   test:
#     runs-on: ubuntu-latest
# Correto:
#   test:
#     runs-on: ubuntu-latest

# ERRO 2: 'python-version' não está indentado dentro de 'with'
# Errado:
#   with:
#     python-version: "3.11"
# Correto:
#   with:
#     python-version: "3.11"

# ERRO 3: os comandos dentro do bloco 'run: |' não estão indentados
# Errado:
#   run: |
#     pip install -r requirements.txt
# Correto:
#   run: |
#     pip install -r requirements.txt

# ERRO 4: sintaxe errada para referenciar variável de ambiente com secret
# Errado: $secrets.HF_TOKEN
# Correto: ${ secrets.HF_TOKEN }}
# (Veremos como configurar secrets corretamente na Parte 3)

# ---
# YAML corrigido:
#
# name: CI Bella Tavola
# on: [push, pull_request]
#
# jobs:
#   test:
#     runs-on: ubuntu-latest
#     steps:
#       - uses: actions/checkout@v4
#
#       - name: Setup Python
```

```
#       uses: actions/setup-python@v5
#       with:
#         python-version: "3.11"
#
#       - name: Instalar dependencias
#         run: |
#           pip install -r requirements.txt
#
#       - name: Rodar testes
#         run: pytest tests/ -v
#         env:
#           HF_TOKEN: ${ secrets.HF_TOKEN }}
```

✓ Exercício 2.3 — Escrevendo o workflow do Bella Tavola

Nível: Essencial

Conceito: Escrever um workflow do zero, colocar o primeiro pipeline no ar.

Antes de começar

Confirme que você completou a verificação local no início do caderno. Em especial, você deve saber responder:

- Qual é o comando exato para iniciar sua API? (`uvicorn main:app` ou outro?)
- Qual rota retorna 200 na sua API? (`/`, `/health`, `/pratos` ou outra?)

Sua tarefa

Crie a pasta `.github/workflows/` na raiz do seu repositório e dentro dela o arquivo `ci.yml`.

O workflow deve:

- Disparar em `push` e `pull_request` para o branch `main`
- Usar `ubuntu-latest` com Python `3.11`
- Executar os seguintes steps em ordem:
 1. Baixar o código
 2. Configurar Python
 3. Atualizar pip e instalar dependências do `requirements.txt`
 4. Iniciar o servidor em background e aguardar 3 segundos
 5. Verificar com `curl` que a API responde na rota que você identificou

Adapte o workflow à estrutura do seu projeto. Os pontos marcados com **(!)** devem ser personalizados:

```
name: CI – Bella Tavola

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  verificar-api:
    runs-on: ubuntu-latest

    steps:
      - name: Baixar o código
        uses: actions/checkout@v4

      - name: Configurar Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Instalar dependências
        run: |
          pip install --upgrade pip
          pip install -r requirements.txt

      - name: Iniciar a API em background
        run: |
          uvicorn __:app &    # (!) ajuste para o caminho do seu app
          sleep 3

      - name: Verificar que a API responde
        run: |
          curl --fail http://localhost:8000/___    # (!) ajuste para sua rota
          echo "API respondeu com sucesso"
```

Fazendo o primeiro deploy do pipeline

1. Faça commit do arquivo **ci.yml**
2. Faça push para o GitHub

3. Abra a aba **Actions** no seu repositório
4. Clique no workflow que apareceu e observe os logs de cada step

✓ Lendo os logs quando algo falha

Quando um step falha, o GitHub marca com **✖** e exibe o log daquele step. Alguns erros comuns e o que significam:

Mensagem no log	Causa provável
<code>ModuleNotFoundError: No module named 'fastapi'</code>	<code>requirements.txt</code> incompleto ou não instalado
<code>Error: No such file or directory: 'main.py'</code>	O arquivo principal tem outro nome ou está em outra pa
<code>uvicorn: command not found</code>	<code>uvicorn</code> não está no <code>requirements.txt</code>
<code>curl: (7) Failed to connect</code>	A API não subiu — verifique o step anterior nos logs
<code>curl: (22) The requested URL returned error: 404</code>	A rota testada não existe — ajuste a URL do curl
<code>Connection refused</code>	A API está em outra porta ou demorou mais de 3s para s

Se a API demorar mais de 3 segundos para iniciar, aumente o `sleep`:

```
- name: Iniciar a API em background
  run: |
    uvicorn main:app &
    sleep 8    # aumente se necessário
```

- #  Anote o que aconteceu após o push:
- # O pipeline passou ou falhou?
- # Se falhou: em qual step apareceu o erro?
- # O que o log dizia exatamente?
- # O que você precisou ajustar?

💡 Gabarito

```
# @title
# Versão de referência do ci.yml — adapte ao seu projeto

# name: CI — Bella Tavola
#
# on:
#   push:
#     branches: [main]
#   pull_request:
```

```
#   branches: [main]
#
# jobs:
#   verificar-api:
#     runs-on: ubuntu-latest
#
#     steps:
#       - name: Baixar o código
#         uses: actions/checkout@v4
#
#       - name: Configurar Python
#         uses: actions/setup-python@v5
#         with:
#           python-version: "3.11"
#
#       - name: Instalar dependências
#         run: |
#           pip install --upgrade pip
#           pip install -r requirements.txt
#
#       - name: Iniciar a API em background
#         run: |
#           uvicorn main:app &
#           sleep 3
#           # 'uvicorn main:app &' → o & coloca o processo em background,
#           # liberando o terminal para o próximo comando.
#           # 'sleep 3' → aguarda a API inicializar antes de testar.
#           # Aumente para 8 ou mais se sua API carrega modelos na inicializaçã
#
#       - name: Verificar que a API responde
#         run: |
#           curl --fail http://localhost:8000/
#           echo "✅ API respondeu com sucesso"
#           # 'curl --fail' → retorna código de saída diferente de 0
#           # se a resposta for um erro HTTP (4xx, 5xx),
#           # fazendo o pipeline falhar corretamente.
#           # Se sua rota raiz não existir, troque '/' por '/health' ou '/prate
```

✓ Exercício 2.4 — Desafio: múltiplos jobs

Nível: Desafio

Conceito: Estrutura com jobs dependentes, separação de responsabilidades no pipeline.

✓ Sua tarefa

Refatore o `ci.yml` para ter **dois jobs separados**:

Job `verificar-sintaxe`:

- Verifica imports não utilizados com `autoflake --check`
- Verifica formatação com `black --check`

Job `verificar-api`:

- Só roda se `verificar-sintaxe` passar (`needs`)
- Sobe a API e verifica se ela responde (igual ao exercício anterior)

Instale as ferramentas necessárias adicionando ao `requirements.txt` (ou instalando no pipeline):

```
pip install autoflake black
```

Comandos de verificação:

```
autoflake --check --remove-all-unused-imports -r .
black --check .
```

Para refletir: Por que faz sentido rodar a verificação de sintaxe antes de subir a API? O que você ganha separando em dois jobs em vez de colocar tudo em um?

```
# ✏️ Escreva seu workflow refatorado no arquivo .github/workflows/ci.yml
# e anote aqui as observações:

# O que acontece na aba Actions quando verificar-sintaxe falha?

# O que acontece com o job verificar-api quando o anterior falha?
```

💡 Gabarito

```
# @title
# name: CI – Bella Tavola
#
# on:
#   push:
#     branches: [main]
#   pull_request:
#     branches: [main]
#
# jobs:
#   verificar-sintaxe:
#     runs-on: ubuntu-latest
```

```
#
# steps:
#   - name: Baixar o código
#     uses: actions/checkout@v4
#
#   - name: Configurar Python
#     uses: actions/setup-python@v5
#     with:
#       python-version: "3.11"
#
#   - name: Instalar ferramentas de qualidade
#     run: pip install autoflake black
#     # Prefira adicionar autoflake e black ao requirements.txt
#     # para garantir que o mesmo ambiente é usado localmente e no CI.
#
#   - name: Verificar imports não utilizados
#     run: autoflake --check --remove-all-unused-imports -r .
#
#   - name: Verificar formatação
#     run: black --check .
#
# verificar-api:
#   runs-on: ubuntu-latest
#   needs: verificar-sintaxe      # só roda se o job anterior passar
#
# steps:
#   - name: Baixar o código
#     uses: actions/checkout@v4
#
#   - name: Configurar Python
#     uses: actions/setup-python@v5
#     with:
#       python-version: "3.11"
#
#   - name: Instalar dependências
#     run: |
#       pip install --upgrade pip
#       pip install -r requirements.txt
#
#   - name: Iniciar a API em background
#     run: |
#       uvicorn main:app &
#       sleep 3
#
#   - name: Verificar que a API responde
#     run: curl --fail http://localhost:8000/
#
# Por que separar em jobs?
# Cada job roda em uma máquina virtual separada e isolada –
# não há estado compartilhado entre jobs, por isso é normal
# repetir checkout e instalação em cada um.
```

```
# A separação permite identificar rapidamente onde o problema está:  
# se verificar-sintaxe falha, o erro fica claro sem misturar  
# com logs da API. E o job verificar-api nem é iniciado,
```

✓ Checklist da Parte 1

Antes de seguir para a Parte 2, confirme que você consegue:

Bloco 1 — CI/CD

- ☐ Explicar a diferença entre CI, CD (Entrega) e CD (Implantação)
- ☐ Identificar situações em que uma mudança pode quebrar a API silenciosamente
- ☐ Classificar práticas reais como CI ou CD

Bloco 2 — GitHub Actions e YAML

- ☐ Explicar a diferença entre `uses` e `run` em um step
- ☐ Identificar os 4 componentes principais de um workflow (`name`, `on`, `jobs`, `steps`)
- ☐ Escrever um gatilho que dispare em push e pull_request
- ☐ Identificar e corrigir erros de sintaxe YAML
- ☐ Adaptar o caminho `main:app` e a rota de teste ao seu projeto
- ☐ Ver os logs de uma execução na aba Actions do GitHub
- ☐ Interpretar as mensagens de erro mais comuns nos logs
- ☐ Fazer o pipeline do Bella Tavola ficar verde ✓